

Índice

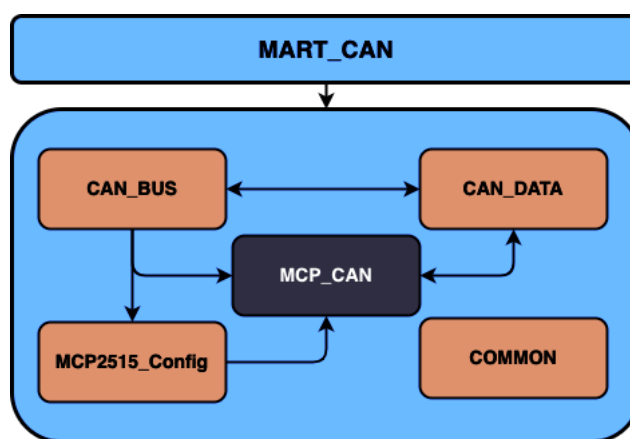
1. Introducción	2
Estructura de MART_CAN.....	2
2. Características principales.....	3
• Envío y recepción de datos:	3
• Procesamiento de datos	3
• Opciones de Configuración	3
• Filtrado de IDs	3
• Opciones de Depuración.....	4
3. Envío y recepción de datos	4
3.1 Recepcion, función receive()	4
3.2 Envío, función send()	4
4. Procesamiento de datos.....	4
4.1 Función getPacket().....	5
4.2 Función setPacket()	5
5. Opciones de configuración	7
5.1 Configuración de paquetes	7
Función setRRFId()	7
Función setPacketTimer().....	7
5.2 Configuración de memoria.....	8
Función setRemovableIds().....	8
Opción autoRemoveRRFPacket	8
Opción autoRemoveStoredFilters.....	9
5.3 Otras configuraciones.....	9
Opción respondToRRF	9
Opción simulating.....	9
6. Filtrado de IDs.....	10
Función setFilters().....	10
Función printFilters().....	10
Función testFilters()	10
7. Depuración	11
Mensajes de estado	11
Mensajes de error	11
8. Bibliografía.....	11

1. Introducción

MART_CAN es una biblioteca diseñada para facilitar y optimizar la comunicación en redes CAN, específicamente para proyectos basados en microcontroladores que utilizan el controlador CAN MCP2515. Se ha empleado el lenguaje C++ y una programación orientada a objetos con la finalidad de que la biblioteca sea versátil y fácilmente escalable.

Otra de las principales consideraciones a la hora de desarrollar la biblioteca ha sido que la implementación sea lo más simple e intuitiva posible, para que el usuario final no se tenga que preocupar de realizar procesamiento o configuraciones extra.

Estructura de MART_CAN



MART_CAN está formada por varios módulos:

- **Biblioteca MCP_CAN**
Biblioteca creada por un usuario de github que es la que se encarga tanto de establecer la comunicación entre el microcontrolador(ESP32) y el módulo MCP2515 por SPI como la del MCP2515 con otros módulos de la red. En resumen, una gestión de los protocolos de comunicaciones a nivel bajo.

El resto de módulos han sido creados por MART para aportar más funcionalidades:

- La clase **CAN_BUS**.
Esta es la clase principal, cuyo principal propósito es el de procesar de manera adecuada la información contenida en memoria para enviar y recibir la información por el bus CAN de manera eficiente y para poder presentarla al usuario final. También gestiona la comunicación por software entre el resto de módulos implementados.
- Dos objetos de la clase **CAN_DATA**
Encargados de almacenar de manera estructurada tanto los datos de entrada como los de salida.

- Un objeto de la clase **MCP2515_Configurator**
Para realizar el filtrado por hardware de los mensajes, entre otros.
- La clase **COMMON**
Que incluye definiciones de estructuras y parámetros globales utilizados por el resto de objetos de la clase MART_CAN.

Ejemplo de inicialización de la biblioteca creando el objeto CAN de la clase CAN_BUS.
El constructor recibe el pin del microcontrolador que está conectado al pin “INT” del MCP2515

```
#include <Arduino.h>
#include "MART_CAN.h"

CAN_BUS CAN(5);
```

2. Características principales

- **Envío y recepción de datos:** MART_CAN facilita el envío y la recepción de datos a través del bus CAN. Utiliza funciones específicas para leer y transmitir paquetes de datos.
 - Función *receive()*: Esta función se encarga de leer los datos del bus CAN y almacenarlos en la memoria de entrada. Puede responder automáticamente a peticiones de datos remota (RRF) si está configurado para ello.
 - Función *send()*: Permite el envío de paquetes de datos almacenados en la memoria de salida. Puede enviar todos los paquetes de manera automática o un paquete específico basado en su ID.
- **Procesamiento de datos:** MART_CAN proporciona métodos para manejar eficientemente los datos de entrada y salida, permitiendo flexibilidad en el tipo de datos transmitidos.
 - Funciones *getPacket()* y *setPacket()*: Estas funciones ayudan en la manipulación de paquetes de datos. *getPacket()* se utiliza para leer datos del bus, mientras que *setPacket()* prepara los datos para su envío.
- **Opciones de Configuración:** Incluye diversas configuraciones para adaptar la comunicación según las necesidades, como la configuración de paquetes RRF y la configuración de la memoria.
- **Filtrado de IDs:** MART_CAN permite configurar máscaras y filtros para controlar qué información se recibe, optimizando así la gestión de datos en la red CAN.

- **Opciones de Depuración:** Proporciona opciones para la depuración y el monitoreo del sistema, facilitando la identificación y resolución de problemas.

3. Envío y recepción de datos

3.1 Recepcion, función *receive()*

La función *receive()* lee los datos que llegan por el bus y los almacena en la memoria de entrada (*DataIN*), para que luego puedan ser procesados y utilizados por el usuario.

Si uno de los paquetes recibido es una petición de datos remota (RRF) y si la opción de respuesta automática está activada ([ver opciones de configuración](#)), buscará el paquete a enviar ([ver configuración de RRF](#)) en la memoria de salida y lo mandará por el bus.

Para recibir paquetes de manera continua será necesario situar la llamada de esta función en el *loop()* antes de intentar leer cualquier paquete de la memoria de entrada.

3.2 Envío, función *send()*

La función *send()* lee los datos almacenados en la memoria de salida (*DataOUT*) y los envía por el bus. Admite dos posibles implementaciones:

- Si es llamada sin argumentos, enviará todos los paquetes almacenados en memoria de manera automática, si estos cumplen con las condiciones necesarias para su envío (ver configuración de paquetes).
- Si es llamada con una ID en el argumento mandará el paquete almacenado en memoria con esa ID (si lo encuentra) de forma inmediata.

Para recibir paquetes de manera continua será necesario situar la llamada de esta función en el *loop()* después de guardar en la memoria de salida los paquetes a enviar.

4. Procesamiento de datos.

A continuación se explica cómo utilizar los métodos de la clase para realizar el procesamiento de datos, tanto los de entrada como los de salida.

Los datos que forman el mensaje a enviar por la red can pueden ser de distinto tipo (enteros, decimales, booleanos, etc..), es por eso que se ha buscado implementar funciones para permitir flexibilidad a la hora de mandar/recibir y no tener que usar una función en concreto para mandar un tipo de dato y otra distinta para mandar un dato que no sea del mismo tipo.

Para que esto sea posible, se han implementado funciones variádicas sobrecargadas con plantillas.

4.1 Función *getPacket()*

getPacket() es llamada después de leer los datos del bus, recibe como argumentos una ID(la del mensaje que se quiere leer que ya está guardado en memoria) y una serie de arrays para almacenar la información de la forma deseada. A continuación se muestran algunos ejemplos de implementación:

```
bool dataBool16[16];
short dataShort1[1],dataShort2[2];
int dataInt1[1],dataInt2[2];
float dataFloat1[1];

//Se leen los datos del bus y se guardan en memoria (DataIN)
CAN.receive();
//Se procesan los primeros 4 bytes del paquete con ID=100 como "int" y se guardan en dataInt1
CAN.getPacket(100, dataInt1);

//Se procesan los 8 bytes del paquete con ID=101 como "int" (de 4 en 4) y se guardan en dataInt2
CAN.getPacket(101, dataInt2);

//Se procesan los primeros 4 bytes del paquete con ID=102 como "short"(de 2 en 2) y se guardan en dataShort2
//Los restantes se procesan como "int" y se guardan en dataInt1
CAN.getPacket(102, dataShort2, dataInt1);

//Se procesan los primeros 2 bytes del paquete con ID=200 como "bool"(de bit en bit) y se guardan en dataBool16
//Los 4 bytes siguientes se procesan como "float" y los dos últimos como "short"
CAN.getPacket(200, dataBool16, dataFloat1,dataShort1);

//getPacket devuelve "false" si se intentan leer más de 8 bytes o si el paquete con la ID buscada no existe
if(!CAN.getPacket(200, dataBool16, dataFloat1,dataShort1,dataInt1))
{
    Serial.println("Error. Se están intentando leer más de 8 bytes");
}
```

Posteriormente y si se desea, se puede utilizar *printArray()* para mostrar la información almacenada:

```
//Muestra la información almacenada en los arrays por pantalla
CAN.printArray(dataInt2);
CAN.printArray(dataBool16);
CAN.printArray(iFloat2);
```

4.2 Función *setPacket()*

setPacket() es llamada antes de mandar los datos al bus, recibe como argumentos una ID(la del mensaje que se quiere guardar en memoria) y una serie de arrays para almacenar la información de la forma deseada. Es posible llamar a la función sólo con la ID como argumento para crear un paquete RRF.A continuación se muestran algunos ejemplos de implementación:



```

bool dataBool16[16] = {1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1};
short dataShort1[1] = {12}, dataShort2[2] = {12, -1148};
int dataInt1[1] = {-3}, dataInt2[2] = {-3, 1234567};
float dataFloat1[1] = {3.142592};

// Se crea un paquete con ID=100 configurado como RRF (petición de datos remota). Los paquetes que estén configurados
// en los receptores responderán enviando información cuando reciban este paquete
CAN.setPacket(100);

// Se procesan los 8 bytes de dataInt2 como "int" (de 4 en 4) y se crea un paquete de ID=101 con esos datos
CAN.setPacket(101, dataInt2);

// Se procesan los 4 bytes de dataShort2 como "short"(de 2 en 2), los 4 bytes de dataInt1 como "int" y se crea
// un paquete de ID=102 con esos datos
CAN.setPacket(102, dataShort2, dataInt1);

// Se crea un paquete que contiene información de tipo "bool", "float" y "short"
CAN.setPacket(200, dataBool16, dataFloat1, dataShort1);

// setPacket devuelve "false" si se intenta crear un paquete de más de 8 bytes
if (!CAN.setPacket(200, dataBool16, dataFloat1, dataShort1, dataInt1))
{
    Serial.println("Error. Se están intentando escribir más de 8 bytes");
}

// Se guardan los datos en memoria DataOUT y se envían por el bus
CAN.send();

```

Otra posible implementación de la función *send()* para reemplazar el envío de datos de manera continua por uno sujeto a unas condiciones de ejecución determinadas (mayor control de flujo):

```

// Se crea un paquete con ID=100 configurado como RRF (petición de datos remota). Los paquetes que estén configurados
// en los receptores responderán enviando información cuando reciban este paquete
CAN.setPacket(100);

// Se procesan los 8 bytes de dataInt2 como "int" (de 4 en 4) y se crea un paquete de ID=101 con esos datos
CAN.setPacket(101, dataInt2);

// Se procesan los 4 bytes de dataShort2 como "short"(de 2 en 2), los 4 bytes de dataInt1 como "int" y se crea
// un paquete de ID=102 con esos datos
CAN.setPacket(102, dataShort2, dataInt1);

// Se crea un paquete que contiene información de tipo "bool", "float" y "short"
CAN.setPacket(200, dataBool16, dataFloat1, dataShort1);

//Lectura de entradas físicas del microcontrolador
bool condicion1 = digitalRead(10);
bool condicion2 = analogRead(14) > 2048;

//Enviar datos por el bus sólo si se cumplen unas condiciones determinadas
if (condicion1) CAN.send(100);
else if (condicion2) CAN.send(101);
else
{
    CAN.send(101);
    CAN.send(200);
}

```

5. Opciones de configuración

Se ha dotado a MART_CAN con varias opciones de configuración con el objetivo de poder adaptar la implementación en función de las necesidades de la comunicación.

5.1 Configuración de paquetes

Función *setRRFId()*

El protocolo CAN permite realizar peticiones de datos remotas a través de mensajes RRF. La función *setRRFId()* permite configurar en el dispositivo receptor qué paquetes previamente almacenados en memoria se van a enviar de forma automática cuando llegue la petición de datos remota (RRF) correspondiente.

```
///Configuración de las asociaciones de paquetes RRF (petición de datos remota)

//Cuando se reciba un paquete RRF con ID=1, se enviarán los paquetes con ID=100 e ID=101 automáticamente
//si están guardados en memoria
CAN.setRRFId(1, 100);
CAN.setRRFId(1, 101);

//Cuando se reciba un paquete RRF con ID=200, ID=201 o ID=202, se enviará el paquete con ID=301 automáticamente
//si está guardado en memoria
CAN.setRRFId(200, 301);
CAN.setRRFId(201, 301);
CAN.setRRFId(202, 301);
```

Función *setPacketTimer()*

Con esta función se pueden configurar determinados paquetes para que se envíen de manera automática cada cierto tiempo.

setPacketTimer() recibe como argumentos una ID y un número entero que determinará el periodo (en milisegundos) de envío del paquete con la ID suministrada. Este envío se hace de manera automática, siempre que se llame a la función *send()*.

Se ha creado esta función para evitar una posible sobrecarga de información en el bus; cada paquete tendrá asignado una frecuencia de envío para controlar el flujo de la información.



```
short dataShort1[1] = {12};
int dataInt1[1] = {-3};

// Se crean dos paquetes con ID=100 e ID=101
CAN.setPacket(100, dataShort1);
CAN.setPacket(101, dataInt1);

// Se crea un RRF para realizar una petición de datos remota
CAN.setPacket(200);

// Se configura el paquete con ID=100 para que sea enviado de manera automática cada 5.5 segundos
CAN.setPacketTimer(100, 5500);

// Se configura el paquete con ID=101 para que sea enviado de manera automática cada 2.0 segundos
CAN.setPacketTimer(101, 2000);

// Se configura el paquete con ID=200 para realizar una petición de datos remota cada 250 ms
CAN.setPacketTimer(200, 250);
```

5.2 Configuración de memoria

Se han implementado varias funcionalidades para optimizar el uso de la memoria, liberando recursos cuando sea necesario:

Función *setRemovableIds()*

Gestiona el borrado automático de datos una vez enviados o recibidos.

Esta función permite seleccionar qué paquetes almacenados en memoria van a ser borrados una vez enviados o recibidos. Recibe como entrada un array que contiene las IDs de los paquetes a eliminar y el número de IDs a eliminar. Si la función se llama sin argumento, entonces todos los paquetes se eliminarán de la memoria una vez procesados.

```
unsigned long removeableIDs[2] = {101, 103};

//Configura la memoria donde se guardan los datos recibidos para que borre todos los paquetes
//que son procesados con getPacket()
CAN.DataIN.setRemovableIds();

//Configura la memoria donde se guardan los datos a enviar para que borre los paquetes con ID=101 e ID=103
//una vez que son enviados con send()
CAN.DataOUT.setRemovableIds(removeableIDs,2);
```

Opción *autoRemoveRRFPacket*

Activada por defecto.

Poner a true para eliminar una petición de datos remota (RRF) una vez que se ha enviado la información solicitada. Si esta opción está activada, será necesario recibir otra RRF para volver a enviar la misma información.



Opción *autoRemoveStoredFilters*

Activada por defecto.

Poner a true para eliminar las máscaras y filtros calculados y guardados en la memoria del microcontrolador una vez que son configurados por hardware en el MCP2515.

```
//Configuración de memoria  
CAN.config.autoRemoveRRFPacket=false;  
CAN.config.autoRemoveStoredFilters=false;
```

5.3 Otras configuraciones

Opción *respondToRRF*

Activada por defecto.

Si está activada, se responderá de manera automática a todas las RRF recibidas

Opción *simulating*

Desactivada por defecto.

Activar opción para simular la presencia de una conexión real de un MCP2515 al microcontrolador si no se tiene ninguno conectado.

```
//Otras configuraciones  
CAN.config.respondToRRF=false;  
CAN.config.simulating=true;
```

Sólo será necesario llamar a las funciones/opciones de configuración una vez, por lo que es recomendable que se sitúen en la función *setup()*

6. Filtrado de IDs

Para evitar que los dispositivos receptores reciban información innecesaria, es posible configurar una serie de máscaras y filtros (en el MCP2515 2 máscaras y 6 filtros máximo). Para facilitar el cálculo de los mismos, MART_CAN cuenta con métodos propios en la clase MCP2515_Configurator para realizar estos cálculos de manera automática.

Función *setFilters()*

Recibe un vector de IDs para calcular de forma automática las máscaras y filtros a aplicar en la configuración interna del MCP2515. Devuelve “true” si la configuración se ha aplicado de manera satisfactoria.

```
//Las únicos mensajes que se recibirán serán aquellos cuyas IDs estén configuradas con setFilters
std::vector<uint16_t> configIds = {0x101, 0x102, 0x109, 0x110};
if (CAN.setFilters(configIds))
{
    Serial.println("Filtros aplicados correctamente");
}
```

Función *printFilters()*

Muestra las máscaras y los filtros calculados por pantalla.

```
//Muestra las máscaras y los filtros calculados previamente por pantalla
CAN.printFilters()
```

Función *testFilters()*

Recibe un vector de IDs y realiza comprobaciones de filtrado por software con las máscaras y filtros creados anteriormente para comprobar el correcto funcionamiento del filtrado mostrando el resultado de aplicar cada una de las IDs recibidas a los filtros por pantalla.

```
//Las únicos mensajes que se recibirán serán aquellos cuyas IDs estén configuradas con setFilters
std::vector<uint16_t> configIds = {0x101, 0x102, 0x109, 0x110};
if (CAN.setFilters(configIds))
{
    Serial.println("Filtros aplicados correctamente");
}

//Si se utiliza la configuración anterior, sólo las IDs 0x101, 0x102, 0x109 y 0x110 deberían pasar el test.
//El resto deben de ser bloqueadas
std::vector<uint16_t> testIds = {0x101, 0x102, 0x109, 0x110, 0x200, 0x211, 0x500, 0x600, 0x999};

//Muestra el resultado del test por pantalla
CAN.testFilters(testIds);
```

Si se desea realizar el test con la función *testIDs*, la opción “*autoRemoveStoredFilters*” debe de estar desactivada.

7. Depuración

Si el funcionamiento del sistema no es el correcto o si se necesita conocer de manera más detallada cómo están trabajando los métodos de la biblioteca, se pueden activar opciones de depuración para que se muestren por pantalla (monitor serie) mensajes de estado o de error. En la clase *common* se encuentra la lógica para gestionar esta funcionalidad.

Mensajes de estado

Los muestran los métodos (funciones internas de las bibliotecas) para indicar el estado de ejecución de los mismos. Para visualizarlos, incluir al inicio del programa:

```
#define DEBUG
```

Mensajes de error

Mensajes de error como fallos de lectura de memoria y errores en el envío o la recepción de datos. Para visualizarlos, incluir al inicio del programa:

```
#define ERROR_LOGGING
```

8. Bibliografía

- Clase MCP_CAN:
https://github.com/coryjfowler/MCP_CAN_lib
- Funcionamiento del protocolo CAN
<https://www.engineersgarage.com/can-protocol-understanding-the-controller-area-network-protocol/#:~:text=The%20Controller%20area%20network%20or,air%20conditioning%2C%20airbags%2C%20etc.>
- Instrucciones de implementación MCP2515-ESP32
<https://www.circuitstate.com/tutorials/what-is-can-bus-how-to-use-can-interface-with-esp32-and-arduino/>
- GitHub con el repositorio del proyecto MART_CAN
<https://github.com/martElectronics/can-library>